WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

“新泛型”前瞻

主讲人：轩加振

泛型实现及仓颉“新泛型”变化



目录

- “新泛型”前瞻 --混合态泛型

```
package hello

func Compare_Int(a: Int, b: Int): Int {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}

func Compare_Float64(a: Float64, b: Float64): Int {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

```
func Compare<T>(a: T, b: T) where T <: Comparable<T> {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}

main() {
    println(Compare<Int64>(10, 11))
    println(Compare<Float64>(10.0, 11.0))
    return 0
}
```

泛型编程的特点在于把**类型**作为参数

单态化泛型（*monomorphization*）

```
func Compare<T>(a: T, b: T) where T <: Comparable<T> {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}

main() {
    println(Compare<Int64>(10, 11))
    println(Compare<Float64>(10.0, 11.0))
    return 0
}
```

```
func Compare_Int(a: Int, b: Int): Int {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

```
func Compare_Float64(a: Float64, b: Float64): Int {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

特点：

- 编译时，单态
  - 不损失性能
  - 编译时间损失
  - 大小无收益
  - 兼容性影响？

锚点：相比“手写”而言

编译时，对于每一个唯一的实例化，替换为具体的单个函数

跨包的时候会发生什么？

跨包场景（非泛型）

非泛型函数定义在包 hello.pkg 中，提供给下游组件使用

src > pkg > compare.cj > Compare_Float64(Float64, Float64): Int64

```
package hello.pkg

public func Compare_Int(a: Int, b: Int): Int {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}

public func Compare_Float64(a: Float64, b: Float64): Int {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

代码升级

```
package hello.pkg

public func Compare_Int(a: Int, b: Int): Int {
    println("start compare")
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

仅函数体内部发生变化

hello.pkg.cjo v1

hello.pkg.so v1

```
println(Compare_Int(10, 11))
println(Compare_Float64(10.0, 11.0))
```

application

hello.pkg.cjo v2

hello.pkg.so v2

application

不需要重新编译

Compare的内容在哪里？

仓颉 Cangjie

跨包场景（单态化泛型）

泛型函数定义在包 hello.pkg 中，提供给下游组件使用

```
package hello.pkg

public func Compare<T>(a: T, b: T) where T <: Comparable<T> {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

代码升级

```
package hello.pkg

public func Compare<T>(a: T, b: T) where T <: Comparable<T> {
    println("start compare")
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

hello.pkg.cjo v1

hello.pkg.so v1

application

```
println(Compare<Int64>(10, 11))
println(Compare<Float64>(10.0, 11.0))
```

仅函数体内部发生变化

hello.pkg.cjo v2

hello.pkg.so v2

application

需要重新编译

Compare的内容在哪里？

运行期泛型

```
func Compare<T>(a: T, b: T) where T <: Comparable<T> {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}

main() {
    println(Compare<Int64>(10, 11))
    println(Compare<Float64>(10.0, 11.0))
    return 0
}
```

特点：

- 运行时，
  - 运行性能有损失
  - 编译时间有明显收益，取决于使用量（标准库泛型比较多）
  - 二进制大小收益，取决于使用量
  - 兼容性影响？

锚点：相比“手写”而言

编译时生成类型信息，并在运行期根据具体是什么类型来进行对应的处理

跨包场景（运行期泛型）

```
package hello.pkg

public func Compare<T>(a: T, b: T) where T <: Comparable<T> {
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

代码升级

```
package hello.pkg

public func Compare<T>(a: T, b: T) where T <: Comparable<T> {
    println("start compare")
    if (a > b) {
        return 1
    } else if (a < b) {
        return -1
    } else {
        return 0
    }
}
```

hello.pkg.cjo v1

hello.pkg.so v1

application

```
println(Compare<Int64>(10, 11))
println(Compare<Float64>(10.0, 11.0))
```

仅函数体内部发生变化

hello.pkg.cjo v2

hello.pkg.so v2

application

不需要重新编译

Compare的内容在哪里？

混合态泛型

完全单态化泛型 → 部分单态化泛型　在包内以及一些特定承诺API稳定的地方会采用单态化泛型

部分运行期泛型　影响兼容性的部分会采用运行期泛型

编译器实现层面

泛型变元约束变化

泛型变元约束可以为
class/struct/基础数据类型
/interface

↓

泛型变元约束仅可以为
class 或 interface 类型

变更前	变更后
`extend<T> A<T> where T <: Int32 {}`	`extend A<Int32> {}`

`class coo<T> where T <: Int32 { ...T... }`	方法1:
	`class coo { ...Int32... }`
	方法2:
	`class coo<T> { ...T... }`
	方法3:
	`interface I {}`
	`extend Int32 <: I {}`
	`class coo<T> where T <: I { ...T... }`

`func foo<T> where T <: Int32 { ...T... }`	方法1:
	`func foo { ...Int32... }`
	方法2:
	`func foo<T> { ...T... }`
	方法3:
	`interface I {}`
	`extend Int32 <: I {}`
	`func foo<T> where T <: I { ...T... }`

泛型静态成员变量的变化

泛型类型的静态成员变量每个实例化类型存在单独一份，并且可以依赖类型变元

泛型类型的静态成员变量共享一份（仅cjnative后端），并且不能依赖类型变元

```
class A<T> {
    static var a: T // error, 静态不能使用泛型变元 T
    static var b: Int64 = 1
}

main() {
    A<Int32>.a = 2
    println(A<Int64>.a) // 2, 不同泛型版本，公用一静态变量
}
```

当前这种行为在“新泛型”下比较直观，原本的行为实现起来复杂性高，性能也不高

其他变化

- 运行性能有损失，叠加的场景可能会有明显劣化，加紧优化中~
- 编译时间改善，标准库大量使用泛型，实测重度场景会有70%改善
- 二进制大小收益，取决于使用量
- 兼容性改善
- cjo大小改善
- 反射能力还在适配当中

锚点：相比“单态化”泛型而言

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，获取最新官方资讯!
(您也可以替换为自己的二维码/联系方式，邀请社区开发者共建项目)

